

Your first macro

Start any OOo component (e.g., Calc), and access **Tools**→**Macros**→**Organize Macros**→**OpenOffice.org Basics**. This opens a macro organiser window. In the pane on the left, expand the current document's name, select the standard, and click on **New** to create a new module. Or, expand the standard, select an existing module and an available macro, and click on **Edit** to modify code.

This opens a basic IDE to write code (Figure 1).

Macros are classified based on the location they are stored in.

- **My Macros:** User written macros common for all documents
- **OpenOffice.org Macros:** A library of macros provided by OOo
- **Document Specific:** These are also user written but on a 'per document' basis and are available to the current document only.

The following is the code for a simple macro to greet people.

```
1 Sub Main
2     MsgBox "Hello World"
3 End Sub
```

Figure 2 shows the editor, where you can enter the above code. When this code is run, a dialogue appears with the message, "Hello World". This macro is available to the present document only. To make it available for all documents place it under 'My Macros'.

This example works for all types of OOo documents. In the upcoming examples, we will concentrate more on Calc documents because there is greater business value for spreadsheet automation.

Running and debugging

We can use different items in the standard toolbar of the Basic IDE to run/debug macros. A simple one is **Run Macro** (shortcut: F5). Other options are **compile**, **stop**, **step into**, **step over**, **step out**, **toggle breakpoint**, **watch**, etc. Macros can be created and maintained from this IDE also (Figure 2).

The following is the second example of a macro to access sheets, cells, etc., in a Calc document:

```
1 Sub Main
2     Dim oBook as Object
3     Dim curSheet as Object
4     Dim oCell as Object
5     Set oBook = ThisComponent
6     MsgBox "No. of sheets = " & oBook.Sheets.count
7     oBook.sheets(0).Name = "hello"
8     curSheet = oBook.Sheets.getByIndex(0)
9     oCell = curSheet.getCellByPosition(0,0)
10    oCell.Value = 10
11 End Sub
```

In the above code:

- Lines 2-4 declare the objects *oBook*, *curSheet*, *oCell*



Figure 1: OOo Basic macro editor



Figure 2: The OOo macro editor

- Line 5 accesses the current document reference through *ThisComponent* and stores it to *oBook*.
- In Line 6 *oBook.Sheets* represents collection of all sheets in the present workbook and then displays the count.
- Line 7 renames first sheet to "Hello".
- Line 8 accesses the same sheet and stores it in *curSheet*.
- Line 9 accesses the A1 cell in the first sheet and stores it in *oCell*.
- Line 10 changes the value of the A1 *oCell* to 10.

Let's consider some more code snippets to learn how spreadsheets can be controlled in different ways by macros.

To insert five rows before the third row in the first sheet, use the following code:

```
oSheet = ThisComponent.Sheets(0)
oSheet.rows.insertByIndex(2,5)
```

To fill range A1:A10 with a sequence, use the following line of code:

```
For i = 1 to 10
    oSheet.getCellByPosition(i-1,0)=i
Next
```

To remove 10th column:

```
oSheet.columns.removeByIndex(9)
```

To count the number of consecutive non-empty cells in the first column:

```
n=oSheet.Columns(0).computeFunction(com.sun.star.sheet.GeneralFunction.COUNT)
```

To search for a key pattern and highlight all cells in red:

```
Sub FindAndRed(Sheet as Object)
```